**10Pearls Shine**

Data Science Internship Program

# AQI Predictor

## TECHNICAL REPORT

Submitted by


**Hamna Khalid**

**Data Science Intern, 10Pearls**

Electrical Engineer


6 September 2026

Project Repository: [hamnakhalid134-coder/pearls_aqi_predictor]

# Abstract

This report documents Pearls AQI Predictor, an end-to-end machine-learning and MLOps project developed to forecast Lahore's US AQI at 24, 48 and 72 hours ahead. The final workflow uses Open-Meteo/CAMS air-quality and weather data, feature engineering in Python, Hopsworks as the managed Feature Store and Model Registry, GitHub Actions for scheduled automation, and a Streamlit/Plotly dashboard for interactive serving and analysis. A failure-safe JSON snapshot was added so the dashboard continues to open when the Hopsworks backend or credentials are unavailable.

The final model-ready dataset contains 26,160 hourly observations covering 3 September 2023 to 27 August 2026. It contains 51 model features plus three direct forecast targets: AQI_24h, AQI_48h and AQI_72h. A chronological 80/20 split was used so future observations were never used to train models evaluated on earlier data.

Ridge Regression, Random Forest and XGBoost were evaluated against a persistence baseline. XGBoost was selected for the 24-hour horizon, while Ridge Regression produced the best final results for 48 and 72 hours. The registered Hopsworks models achieved MAE values of 17.076, 23.323 and 24.484 respectively, beating persistence MAE at all three horizons. The project also implements EDA visualizations, AQI health classification, SHAP explainability for the 24-hour XGBoost model, GitHub Codespaces development, and a fallback serving path.


**PROJECT STATUS AT REPORT PREPARATION**

The Streamlit application, Hopsworks Feature Store/Model Registry integration, hourly GitHub Actions pipeline, registered models, prediction pipeline and fallback snapshot are implemented. A temporary Hopsworks API-key authentication failure was observed during final testing; the fallback path behaved correctly and kept the dashboard available. FastAPI and daily retraining automation are listed as final integration items rather than claimed as completed.

# Contents

# 1. Introduction

Air pollution is a major environmental and public-health concern in Lahore. AQI forecasts can provide an early indication of worsening conditions and can support planning, awareness and health-oriented alerts. The purpose of this internship project was not only to train a regression model, but to build a repeatable cloud-based pipeline from data acquisition to feature storage, model registration, automated prediction and a public-facing dashboard.

The project was intentionally designed around free or serverless services. This made the engineering problem broader than model selection: the system had to acquire data automatically, maintain historical features, use reproducible train/test logic, register models, survive cloud service failures and expose understandable forecasts.

# 2. Problem Statement

Develop an end-to-end machine-learning system that predicts Lahore's Air Quality Index 24, 48 and 72 hours into the future using weather and pollutant data. The solution must support historical backfilling, automated feature generation, managed feature/model storage, objective model comparison, explainability and an interactive dashboard while remaining suitable for serverless or free-tier deployment.

# 3. Project Objectives

- Collect hourly air-quality and meteorological data for Lahore.
- Build a multi-year historical dataset instead of relying on a short 90-day window.
- Engineer time, lag, rolling and pollution/weather features.
- Create three explicit future targets for +24 h, +48 h and +72 h AQI.
- Store engineered features in Hopsworks and register production models in its Model Registry.
- Compare multiple regression models with MAE, RMSE and $R^2$ against a persistence baseline.
- Automate the hourly live-feature pipeline with GitHub Actions.
- Build a professional Streamlit dashboard with forecasts, EDA, health categories and model metrics.
- Use SHAP to explain the +24 h XGBoost forecast when the live model is available.
- Keep the dashboard usable during Hopsworks outages or authentication failures using a last-successful snapshot.

# 4. Technology and Software Tools Used

| Technology / Tool | Use in the Project |
|---|---|
| Python | Data acquisition, preprocessing, feature engineering, training and serving logic |
| Pandas / NumPy | Tabular processing and numerical feature creation |
| Scikit-learn | Ridge Regression, Random Forest, metrics and preprocessing |
| XGBoost | Gradient-boosted tree model selected for +24 h forecasting |

| | |
|---|---|
| Hopsworks | Feature Store and Model Registry |
| GitHub | Version control and source repository |
| GitHub Codespaces | Cloud development environment used for implementation and testing |
| GitHub Actions | Hourly scheduled feature-pipeline automation |
| Streamlit | Interactive AQI dashboard |
| Plotly | Interactive charts, AQI gauge and analytical visualizations |
| SHAP | Local model explainability for the XGBoost model |
| Open-Meteo / CAMS | Air-quality and weather observations used by the final V2 pipeline |
| Joblib | Serialized model loading for registry artifacts |
| JSON fallback snapshot | Last-successful prediction path when Hopsworks is unavailable |

# 5. Data Sources and Dataset

## 5.1 Geographic Scope

Lahore, Pakistan was used throughout the project. The working coordinates were latitude 31.5204 and longitude 74.3587.

## 5.2 Data Source

The final V2 workflow uses Open-Meteo services for air-quality and weather variables. The dashboard identifies the source as Open-Meteo/CAMS. Variables used across the project include AQI, PM2.5, PM10, CO, NO, NO2, O3, SO2, NH3, temperature, humidity, pressure, precipitation and wind speed, subject to availability in the relevant pipeline stage.

## 5.3 Final Model-Ready Dataset

| Property | Final V2 Value |
|---|---|
| Rows | 26,160 |
| Date range | 03 Sep 2023 00:00 to 27 Aug 2026 23:00 |
| Total columns | 55 |
| Model features | 51 |
| Targets | AQI_24h, AQI_48h, AQI_72h |
| Missing values in final model-ready table | 0 |
| Duplicate timestamps in final model-ready table | 0 |
| Evaluation split | Chronological 80% train / 20% test |

The chronological split is important because random shuffling would allow temporal information from later observations to leak into training. The local split contained approximately 20,856 training rows and 5,232 test rows. The Hopsworks feature-view read produced a one-row difference in each partition during cloud training, but the time ordering and evaluation principle remained unchanged.

# 6. Methodology

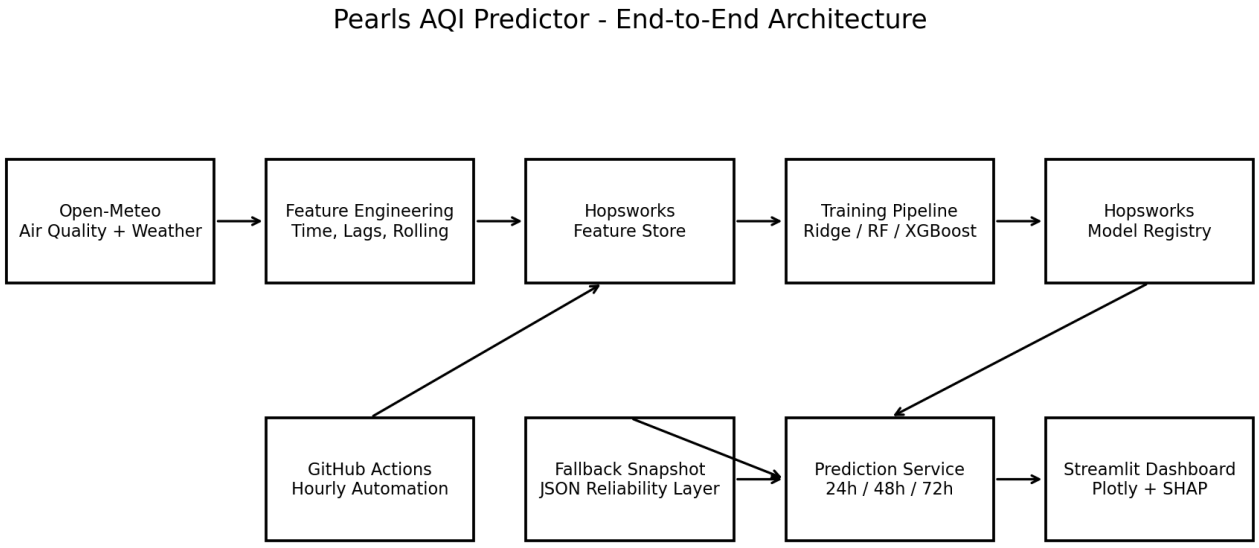


*Figure 1 - End-to-end architecture of Pearls AQI Predictor.*

| Stage | Implementation |
|---|---|
| Historical backfill | Multi-year hourly air-quality and weather observations were acquired for Lahore. |
| Merge and cleaning | Air-quality and weather observations were aligned by timestamp, duplicates removed and missing values checked. |
| Feature engineering | Time, cyclic, lag, rolling and change features were produced. Three future AQI targets were created. |
| Feature Store | The final engineered historical data was uploaded to Hopsworks as a historical feature group. A separate live feature group stores current hourly features. |
| Model training | Ridge, Random Forest and XGBoost were trained and evaluated per forecast horizon. |
| Model selection | The model with the strongest test performance at each horizon was selected, while persistence remained the reference baseline. |
| Model Registry | The chosen models were trained in the Hopsworks-connected pipeline and registered as versioned artifacts. |
| Hourly serving | The scheduled live pipeline writes fresh features; the prediction script reads the newest live feature row and applies the registered models. |
| Dashboard | Streamlit presents AQI status, 24/48/72-hour forecasts, performance tables, EDA charts and SHAP when available. |
| Fallback | If Hopsworks cannot be reached, the application reads latest_prediction_snapshot.json and remains operational. |

# 7. Feature Engineering

The final model input contains 51 engineered features. The feature set combines current pollutant and weather measurements with temporal structure and historical AQI behaviour. The purpose is to let the models use both current environmental conditions and recent dynamics.

| Feature Group | Purpose |
|---|---|
| Time features | Hour, day-related variables and cyclic encodings where applicable |
| Current air-quality features | AQI and pollutant concentrations such as PM2.5, PM10, NO2, O3, SO2 and CO |
| Weather features | Temperature, humidity, pressure, precipitation and wind speed |
| Lag features | Historical AQI/pollutant/weather values at earlier hours |
| Rolling features | Rolling averages/statistics over recent windows |
| Change features | Recent changes in AQI and related variables |
| Targets | AQI exactly 24, 48 and 72 hours ahead |

The first valid rolling observations appear only after enough historical context exists. Rows without complete engineered inputs or future targets are excluded from the final training table.

# 8. Hopsworks Feature Store and Model Registry

## 8.1 Feature Groups

| Hopsworks Object | Role |
|---|---|
| aqi_historical_features v1 | 26,160 model-ready historical rows used for training and analysis |
| aqi_live_features v1 | Latest hourly engineered records used by the prediction application |

Hopsworks provides a managed Feature Store, separating feature engineering from downstream training and serving. This avoids embedding the entire historical dataset directly inside the web application and makes the pipeline closer to a production MLOps architecture.

## 8.2 Registered Models

| Registry Model | Version | Forecast Horizon |
|---|---|---|
| aqi_24h_xgboost | 1 | +24 h |
| aqi_48h_ridge | 1 | +48 h |
| aqi_72h_ridge | 1 | +72 h |

The training script train_and_register.py reads the feature views, trains the selected algorithms and registers the resulting model artifacts in Hopsworks. The prediction path downloads the registry artifact, loads the serialized model with joblib and applies the model's expected feature order.

# 9. Models Used and Comparative Evaluation

## 9.1 Models Evaluated

| Model | Rationale |
|---|---|

| | |
|---|---|
| Persistence baseline | Assumes future AQI will remain equal to the current value. It is a strong reference for autocorrelated time series. |
| Ridge Regression | Linear regression with L2 regularization. Stable and computationally efficient. |
| Random Forest Regressor | Ensemble of decision trees that captures nonlinear relationships and interactions. |
| XGBoost Regressor | Gradient-boosted decision trees optimized sequentially to reduce prediction error. |

## 9.2 Final Registered Model Performance

| Horizon | Selected Model | MAE | RMSE | $R^2$ | Persistence MAE | MAE Improvement |
|---|---|---|---|---|---|---|
| +24 h | XGBoost | 17.076 | 23.580 | 0.627 | 20.228 | 15.6% |
| +48 h | Ridge Regression | 23.323 | 31.637 | 0.329 | 24.848 | 6.1% |
| +72 h | Ridge Regression | 24.484 | 32.868 | 0.277 | 26.500 | 7.6% |

XGBoost produced the strongest final +24 h result, with MAE 17.076 and $R^2$ 0.627. For the longer 48 h and 72 h horizons, Ridge Regression generalized better than the tree models. This confirms that a single algorithm was not forced across all horizons; each forecast horizon was treated as a separate regression problem.

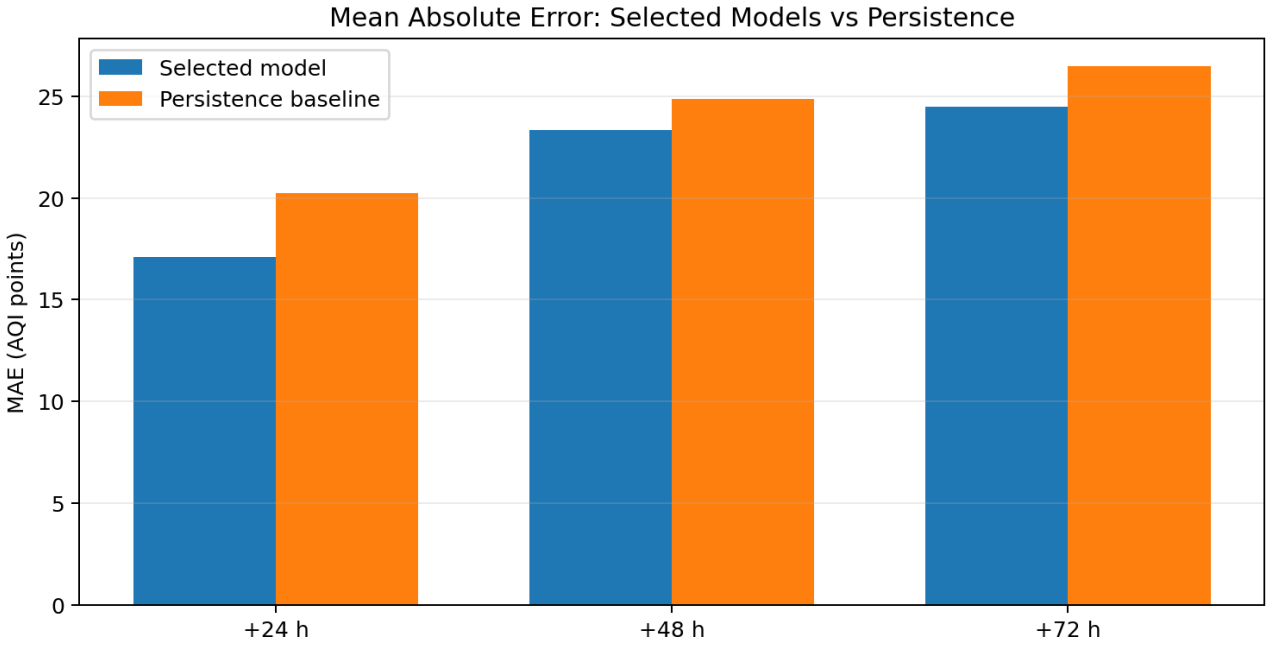


*Figure 2 - MAE of the selected model versus the persistence baseline.*

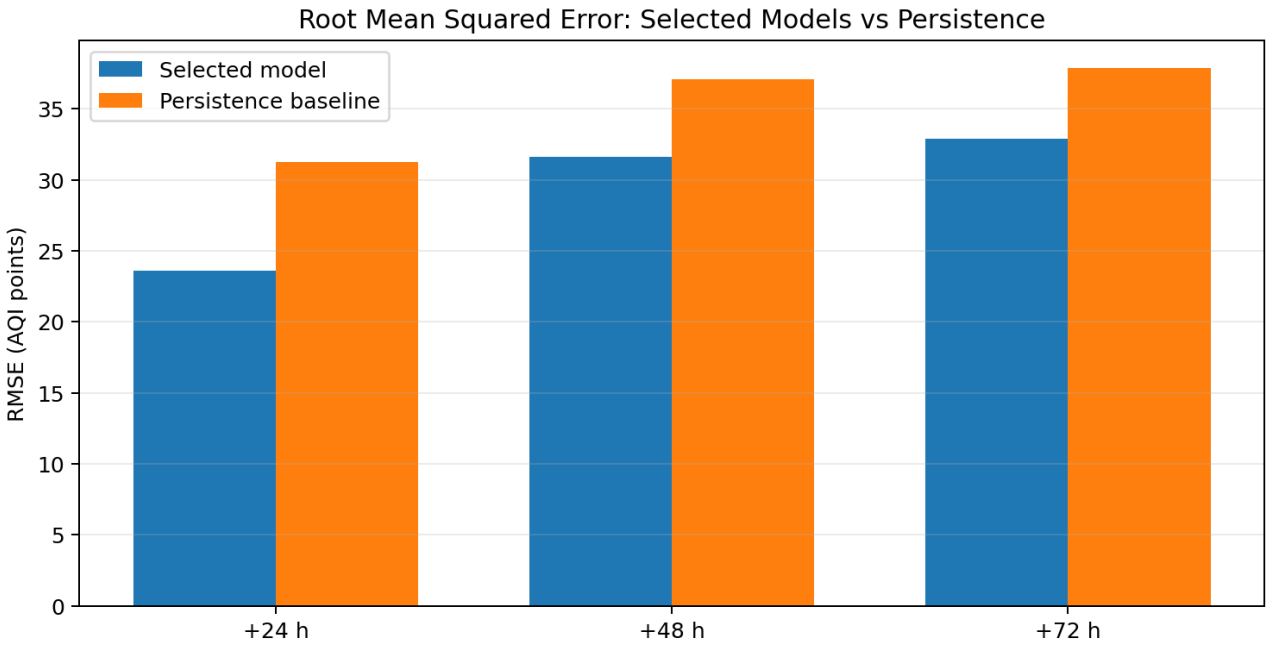


*Figure 3 - RMSE of the selected model versus the persistence baseline.*

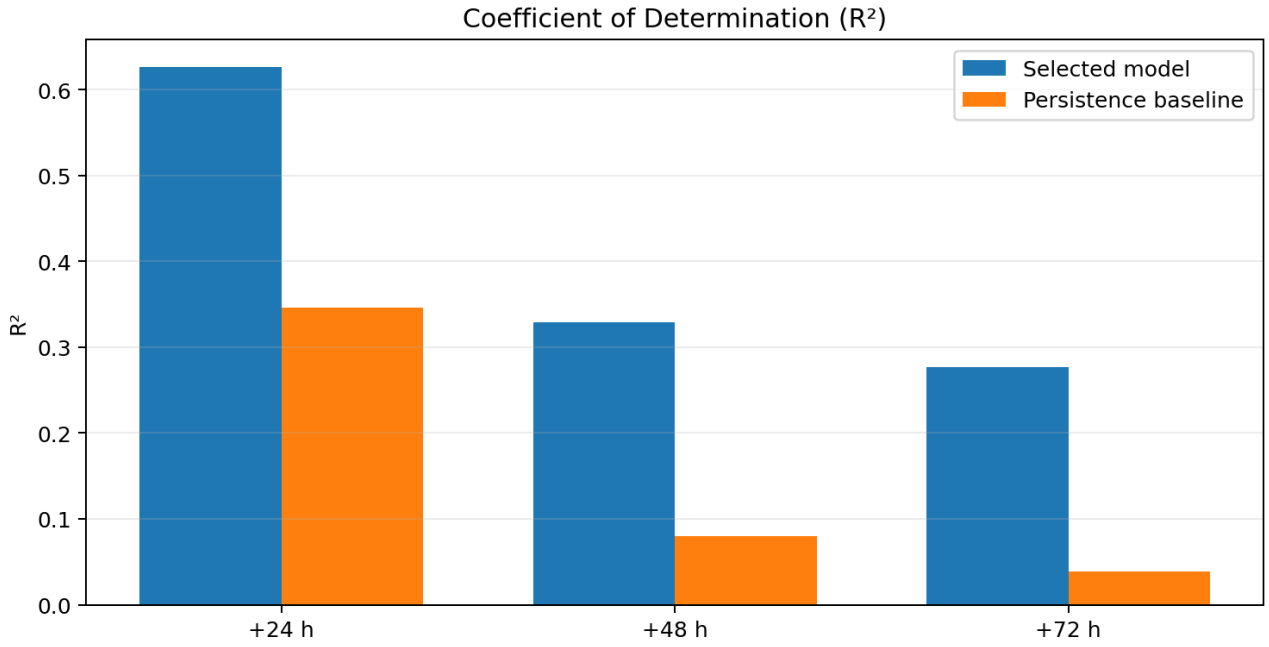


*Figure 4 - $R^2$ of the selected model and persistence baseline.*

## 10. Exploratory Data Analysis

EDA was included both during model development and in the final dashboard. The application fetches recent Open-Meteo air-quality history and visualizes observed AQI, a three-day forecast trajectory, mean AQI by hour of day and current pollutant concentrations. These views make it possible to inspect recent pollution behaviour rather than presenting model outputs as isolated numbers.

| EDA / Visualization | Interpretation |
|---|---|
| Observed AQI time series | Shows recent temporal variation and places the forecast in context. |

| | |
|---|---|
| Mean AQI by hour of day | Examines whether pollution follows a repeatable daily cycle. |
| Current pollutant concentrations | Displays PM2.5, PM10, NO2 and O3 values used to interpret current air quality. |
| MAE / RMSE comparison | Shows whether selected models improve on the persistence baseline. |
| $R^2$ comparison | Shows explained variance by horizon. |
| AQI gauge and EPA category | Maps the numeric AQI to an interpretable health category. |

The most important model-level EDA result is the decrease in predictive strength with horizon. The +24 h model has $R^2$ 0.627, while +48 h and +72 h fall to 0.329 and 0.277. This is expected: uncertainty grows as the forecast moves further from the latest observed conditions.

# 11. SHAP Model Explainability

SHAP (SHapley Additive exPlanations) is implemented for local explanation of the +24 h XGBoost model. In live mode, the dashboard constructs a TreeExplainer for the registered XGBoost model and calculates a SHAP value for each current input feature.

A positive SHAP value means that the feature pushes the predicted AQI upward relative to the model's reference value; a negative value pushes the prediction downward. The dashboard sorts features by absolute importance and displays the top contributors as a horizontal bar chart.

**FALLBACK BEHAVIOUR FOR EXPLAINABILITY**
The fallback JSON stores the last successful predictions and atmospheric values, not the full XGBoost model object. Therefore SHAP is intentionally unavailable in fallback mode. This is safer than presenting a stale or fabricated explanation. The dashboard continues to show the forecast and explicitly marks SHAP as temporarily unavailable.

# 12. GitHub Codespaces and GitHub Actions

## 12.1 GitHub Codespaces

Development was carried out in a GitHub Codespace using Python 3.12. Codespaces provided a reproducible cloud environment for running Hopsworks integrations, Streamlit, feature scripts and Git commands without depending on a local machine configuration.

## 12.2 Hourly Feature Pipeline

The repository contains .github/workflows/hourly_features.yml. The workflow is scheduled hourly and can also be started manually with workflow_dispatch. It checks out the repository, sets up Python 3.12, installs requirements, reads HOPSWORKS_API_KEY from GitHub Actions Secrets and runs hourly_feature_pipeline.py.

**VERIFIED AUTOMATION**
The hourly GitHub Actions workflow was manually executed successfully during project development. This confirms that the live feature pipeline can run independently of the interactive Codespace session.

## 13. Failure-Safe Fallback Design

Cloud services and credentials can fail even when model code is correct. A recruiter or evaluator should still be able to open the application, so the final dashboard was modified to use a two-stage serving strategy.

| Serving Condition | Behaviour |
|---|---|
| Hopsworks available | Read latest live feature row, load registered models, compute fresh +24/+48/+72 forecasts, enable SHAP. |
| Hopsworks unavailable / invalid key / cloud failure | Load latest_prediction_snapshot.json, show the last successful AQI and forecasts, display a visible fallback notice, keep the dashboard usable. |
| Both Hopsworks and snapshot unavailable | Show a controlled backend error instead of silently displaying invalid results. |

During final testing, Hopsworks returned HTTP 401 because the session API key was no longer valid. The fallback activated automatically and the Streamlit dashboard remained available. This was a real failure test rather than a simulated success case.

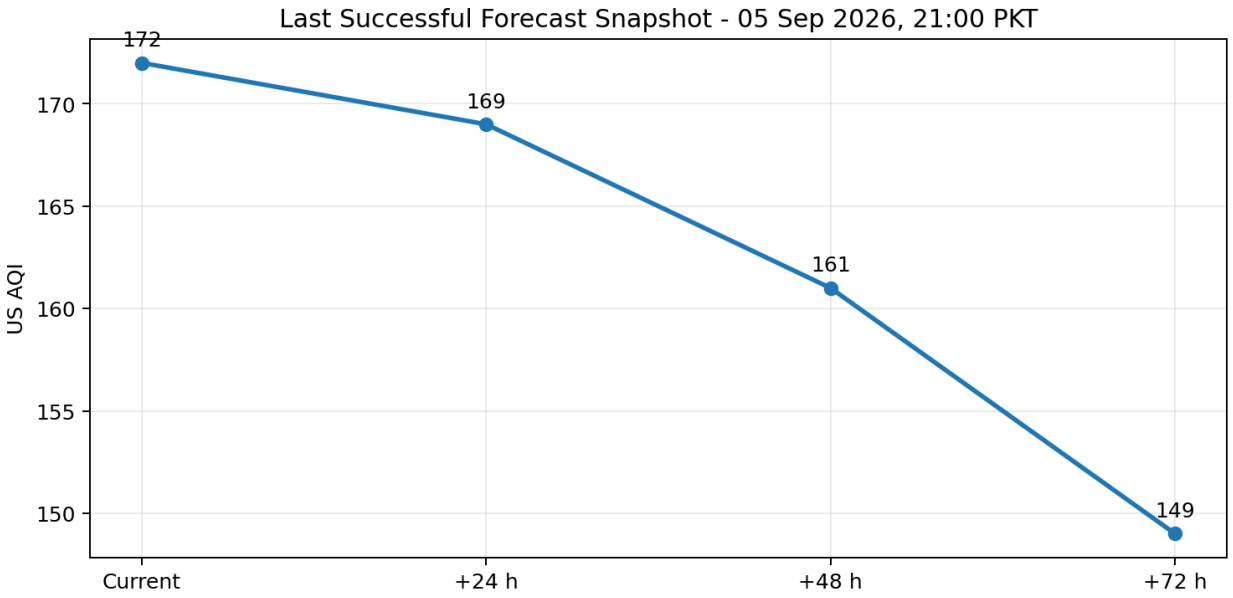


*Figure 5 - Last successful fallback snapshot used during the final Hopsworks authentication failure.*

The snapshot shown above contained current AQI 172 and forecasts of 169 (+24 h), 161 (+48 h) and 149 (+72 h) from the last successful reading at 05 September 2026, 21:00 PKT.

## 14. Streamlit Dashboard and Serving

The front end is implemented in Streamlit with a professional light atmospheric theme. The main screen is organized around current conditions, health interpretation and the three forecast horizons.

| Dashboard Component | Purpose |
|---|---|
| Current US AQI gauge | Numeric AQI with US EPA category and intensity scale |
| Current atmospheric conditions | Temperature, humidity, wind speed, PM2.5, PM10 and O3 |
| Additional observations | Pressure, precipitation, NO2, SO2 and CO |
| Three-Day AQI Forecast | Separate +24 h, +48 h and +72 h prediction cards |

| | |
|---|---|
| 72-Hour Forecast Outlook | Summarizes the maximum forecast and whether conditions are improving or worsening |
| Model Performance Benchmark | MAE, RMSE, $R^2$ and persistence comparison |
| Forecast Analytics | Recent observed AQI, model forecast, hourly pattern and pollutants |
| SHAP | Top local feature contributions for +24 h XGBoost in live mode |
| Project links | GitHub, LinkedIn and technical-report links |

The dashboard's AQI categories follow the displayed US EPA classification ranges: Good 0-50, Moderate 51-100, Unhealthy for Sensitive Groups 101-150, Unhealthy 151-200, Very Unhealthy 201-300 and Hazardous 301+.

# 15. Results and Discussion

The strongest forecast is the 24-hour XGBoost model. It improves MAE over persistence by 15.6% and reduces RMSE from 31.224 to 23.580. Performance falls at longer horizons, but both Ridge models still improve MAE and RMSE over persistence.

The results also show why horizon-specific selection was important. XGBoost's nonlinear capacity was useful at +24 h, while Ridge Regression was more stable for +48 h and +72 h. This indicates that the relationship between current features and future AQI becomes weaker and noisier with time.

The project should be judged as an MLOps system rather than only by a single accuracy number. It includes historical backfill, a Feature Store, Model Registry, scheduled cloud workflow, model comparison, explainability, a user-facing dashboard and a tested fallback path.

# 16. Limitations

- The project currently forecasts one city only: Lahore.
- Open-Meteo/CAMS values are modeled air-quality data and may differ from a specific ground monitoring station or consumer AQI application.
- Forecast uncertainty increases strongly at 48 and 72 hours, as reflected by lower $R^2$.
- The fallback snapshot preserves availability but is not a substitute for fresh live inference; its timestamp must be shown clearly.
- SHAP is unavailable in snapshot fallback mode because the model object is not loaded.
- At final report preparation, the Hopsworks API key used in the Codespace session returned 401 Unauthorized and required rotation.
- Daily retraining automation and a separate FastAPI service are final integration items unless completed before submission.

# 17. Future Work

1. Rotate and centralize Hopsworks credentials in Streamlit Cloud and GitHub Actions secrets.
2. Add a daily model-training workflow and make the dashboard automatically resolve the latest registry model version.
3. Add a minimal FastAPI service with /health and /predict endpoints for programmatic access.
4. Create an automatically refreshed fallback snapshot instead of relying on a manually generated last-successful file.
5. Evaluate a TensorFlow neural network on the same final V2 dataset so the deep-learning comparison is directly comparable.
6. Extend the project to additional Pakistani cities and compare city-specific models.
7. Add prediction intervals or probabilistic AQI exceedance alerts rather than only point forecasts.

# 18. Requirement Coverage

| Requirement | Project Status |
|---|---|
| Raw weather + pollutant API | Completed - Open-Meteo/CAMS |
| Time and derived feature engineering | Completed - 51 final features |
| Feature Store | Completed - Hopsworks historical and live feature groups |
| Historical backfill | Completed - multi-year final model-ready dataset |
| Training and evaluation | Completed - Ridge, Random Forest, XGBoost + persistence |
| MAE, RMSE, $R^2$ | Completed - all three horizons |
| Model Registry | Completed - 3 registered Hopsworks models |
| Hourly automation | Completed - GitHub Actions |
| Daily training automation | Pending final integration at report preparation |
| Streamlit dashboard | Completed |
| FastAPI / Flask service | Pending final integration at report preparation |
| EDA | Completed in dashboard and model-development workflow |
| SHAP / LIME | Completed - SHAP for +24 h XGBoost in live mode |
| Hazardous AQI classification | Completed - US EPA category display and health messages |
| Failure-safe serving | Completed - JSON snapshot fallback |
| Technical report | Completed - this document |

# 19. Reproducibility

Repository:

`https://github.com/hamnakhalid134-coder/pearls_aqi_predictor`

Typical local/Codespaces commands:

```
pip install -r requirements.txt
python hourly_feature_pipeline.py
python predict_aqi.py
python -m streamlit run app.py --server.address 0.0.0.0 --server.port 8501
```

Sensitive credentials are not stored directly in source code. HOPSWORKS_API_KEY is supplied through environment variables, GitHub Actions Secrets and, for deployment, should be supplied through Streamlit Secrets.

# 20. Conclusion

Pearls AQI Predictor demonstrates a complete transition from raw environmental data to a cloud-oriented forecasting application. The final system produces independent 24, 48 and 72-hour AQI forecasts, stores features and models in Hopsworks, automates live feature updates through GitHub Actions, and presents the outputs through a professional Streamlit dashboard.

The best final models were XGBoost for +24 h and Ridge Regression for +48 h and +72 h. All three selected models improved MAE over the persistence baseline. Just as importantly, the final application was designed to fail visibly and safely: when Hopsworks authentication failed during final testing, the snapshot fallback kept the application available rather than producing a blank dashboard.

The project therefore addresses both predictive modeling and operational reliability. Remaining work is limited to final deployment/integration tasks rather than the core data, model or dashboard workflow.

# 21. References


1. Open-Meteo. Weather Forecast and Air Quality APIs. https://open-meteo.com/
2. Hopsworks. Feature Store and Model Registry documentation. https://www.hopsworks.ai/
3. GitHub. GitHub Actions and Codespaces documentation. https://docs.github.com/
4. Streamlit. Streamlit documentation. https://docs.streamlit.io/
5. Plotly. Python graphing library documentation. https://plotly.com/python/
6. Lundberg, S. M., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. NeurIPS.
7. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. KDD.
8. Scikit-learn documentation. Ridge Regression, Random Forest Regressor and regression metrics.